



UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

INGENIERÍA EN DESARROLLO Y GESTION DE SOFTWARE

ASIGNATURA: DESARROLLO WEB PROFESIONAL

8º CUATRIMESTRE GRUPO “A”

ANIMACIONES ACCESIBLES Y OPTIMIZACIÓN DE PERFORMANCE

PROFR. JOSE MIGUEL CARRERA PACHECHO

LILIA HERNÁNDEZ TUN

Animaciones accesibles y optimización de performance

Característica	Animaciones CSS (@keyframes / transition)	Animaciones JavaScript (WAAP / GSAP)
Dificultad	Baja. Sintaxis declarativa, rápida de implementar.	Media/Alta. Requiere lógica, control de estado y manejo de eventos.
Modelo de ejecución	Declarativo. El navegador interpreta reglas de estilo.	Imperativo. El desarrollador controla el flujo de ejecución.
Control	Limitado. Pausar o alterar en tiempo real requiere trabajo adicional.	Total. Permite pausar, revertir, modificar duración y sincronizar múltiples animaciones.
Rendimiento	Excelente cuando se anima transform y opacity (usa el compositor).	Variable. Puede verse afectado si el hilo principal está ocupado.
Uso del hilo	Generalmente ejecutado en el <i>compositor thread</i> .	Depende del hilo principal, salvo optimizaciones internas.
Dependencias	Ninguna. Es parte del estándar CSS.	Puede usar la Web Animations API (nativa) o librerías como GSAP .
Interactividad	Limitada a cambios de clase, pseudo-clases (:hover, :focus).	Dinámica. Puede reaccionar a eventos, datos externos o scroll.
Secuencias complejas	Difícil de escalar si hay múltiples timelines.	Ideal para timelines encadenados y animaciones sincronizadas.
Animaciones basadas en física	No soportadas nativamente.	Soportadas mediante librerías especializadas.
Mantenimiento	Separación clara entre estilos y lógica.	Puede mezclarse con la lógica de negocio si no se estructura bien.
Peso del proyecto	No aumenta el bundle.	Puede aumentar el tamaño si se usan librerías externas.
Uso recomendado	Microinteracciones, botones, loaders, transiciones simples.	Dashboards interactivos, juegos web, storytelling, apps SPA complejas.

¿Cuándo una animación mejora la experiencia y cuándo la perjudica?

Las animaciones mejoran la experiencia cuando:

1. **Guían al usuario visualmente:** Ayudan a entender cambios de estado o relaciones entre elementos.
2. **Proporcionan retroalimentación clara:** Pequeños efectos al interactuar (como un botón que crece o cambia color al presionar) confirman que la acción fue registrada.
3. **Facilitan la navegación y transición entre estados:** Una transición suave entre pantallas o secciones crea una sensación de fluidez y continuidad.
4. **Hacen esperas más tolerables:** Animaciones de carga o microinteracciones bien diseñadas hacen que procesos largos se sientan más cortos y controlados.
5. **Refuerzan la identidad visual de la marca:** Pueden hacer que la interfaz se sienta más cuidada, coherente y memorable.

Las animaciones dejan de ser útiles cuando distraen, dificultan la interacción o impactan técnicamente la página, por ejemplo:

1. **Son demasiado largas o lentas:** Si la animación retrasa acciones comunes.
2. **Hay demasiada animación en la página:** Muchas animaciones simultáneas o efectos innecesarios compiten por la atención y crean sobrecarga visual.
3. **Afectan rendimiento o tiempos de carga:** Animaciones complejas, especialmente con JavaScript pesado, pueden hacer que la interacción se sienta errática (“jank”) o lenta en dispositivos modestos.
4. **No respetan preferencias de accesibilidad:** Algunas personas tienen sensibilidad al movimiento; animaciones rápidas, parpadeos o efectos complejos pueden causar malestar, distracción o incluso síntomas físicos.
5. **Carecen de propósito funcional:** Decoraciones llamativas sin un propósito claro no contribuyen a la comprensión o uso de la interfaz.

¿Qué es prefers-reduced-motion y por qué es importante?

Es una función de medios CSS (Media Query) que permite a los desarrolladores detectar si el usuario ha activado una configuración en su sistema operativo para minimizar la cantidad de movimiento no esencial en la pantalla.

No es solo una cuestión de "gusto visual", sino de salud y usabilidad:

- Trastornos Vestibulares: Muchas personas sufren de mareos, vértigo o náuseas ante movimientos rápidos o inesperados en la pantalla.
- Epilepsia Fotosensible: Ciertos tipos de parpadeos o movimientos rítmicos pueden desencadenar ataques.
- Distracción Cognitiva: Usuarios con TDAH o condiciones similares pueden encontrar que las animaciones constantes les impiden concentrarse en el contenido principal.
- Rendimiento: En dispositivos de gama baja, reducir las animaciones ahorra batería y memoria RAM.

¿Cómo afectan las animaciones al rendimiento de una aplicación web?

1. El Hilo Principal vs. El Hilo del Compositor: El navegador tiene un "hilo principal" (*Main Thread*) donde ejecuta JavaScript y calcula el diseño. Si una animación corre aquí y el JS es pesado, la animación se verá entrecortada (Jank).
2. Las 3 etapas del impacto (Costo de rendimiento): Dependiendo de la propiedad CSS que animes, el navegador debe repetir ciertos pasos. Entre más pasos repita, peor es el rendimiento:
 - a. **Layout (Diseño):** El paso más caro. El navegador recalcula la geometría y posición de los elementos.
 - *Propiedades que lo activan:* width, height, top, left, margin, padding.
 - Impacto: Muy alto (evitar animar estas propiedades).
 - b. **Paint (Pintado):** El navegador rellena los píxeles (colores, sombras, bordes).
 - *Propiedades que lo activan:* background-color, box-shadow, color.
 - Impacto: Medio (más barato que el Layout, pero sigue consumiendo CPU).

3. **Composite (Composición):** El paso más barato. El navegador solo junta las capas ya pintadas. Se procesa en la GPU.
 - *Propiedades que lo activan:* transform (scale, rotate, translate) y opacity.
 - Impacto: Bajo (Es el estándar de oro para animaciones fluidas a 60 FPS).

3. El concepto de "Jank" y los 60 FPS

Para que una animación se sienta "mantequilla" (fluida), el navegador debe generar 60 cuadros por segundo (FPS). Esto significa que el navegador tiene solo 16.6ms para procesar cada cuadro. Si una animación de width tarda 20ms en calcular el Layout, el usuario verá un salto visual o "lag".

4. Consumo de Batería y CPU

Las animaciones mal optimizadas mantienen la CPU/GPU trabajando al máximo. En dispositivos móviles, esto se traduce en:

- Sobrecalentamiento del dispositivo.
- Drenado rápido de la batería.
- Lentitud general en el resto de la aplicación.

¿Qué errores comunes existen al animar interfaces?

1. Errores de Rendimiento (Performance)
 - Animar propiedades de Layout: Usar top, left, width o margin. Esto obliga al navegador a recalcular la posición de todos los elementos en cada cuadro (Frame), causando Jank (lag visual).
 - No usar will-change con moderación: Esta propiedad le avisa al navegador que un elemento se va a mover para que se prepare. El error es ponérselo a todos los elementos, lo que agota la memoria de la tarjeta gráfica (GPU).
2. Errores de Accesibilidad (A11y)
 - Ignorar prefers-reduced-motion: No dar una alternativa estática o suave a personas con trastornos vestibulares o epilepsia.

- Bloqueo del foco del teclado: Animar la entrada de un modal pero permitir que el usuario pueda seguir navegando con el teclado por los elementos que están "detrás" y ocultos.
- Flash o parpadeos excesivos: Movimientos rápidos (más de 3 veces por segundo) que pueden inducir ataques en personas con fotosensibilidad.

3. Errores de Experiencia de Usuario (UX)

- Animaciones demasiado lentas: Una transición de más de 500ms suele desesperar al usuario. La animación debe acompañar la acción, no retrasarla.
- Falta de propósito (Decoración pura): Animar elementos solo "porque se ve bien" sin que aporten feedback o guíen la vista. Esto genera fatiga visual.
- No poder interrumpir la animación: Obligar al usuario a ver toda una secuencia antes de poder hacer clic en un botón. La interfaz debe responder de inmediato.

REFERENCIAS

- Google Developers. (s.f.). Comparación entre las animaciones de CSS y JavaScript. web.dev. <https://web.dev/articles/animations-guide?hl=es-419>
- We-School. (2023). CSS vs JavaScript para animaciones: ¿Cuál elegir para tu proyecto? <https://we-school.es/css-vs-javascript-para-animaciones/>
- Aprender Marketing. (s.f.). Sin movimiento no siempre significa "prefers-reduced-motion": Trucos CSS para accesibilidad. <https://aprendermarketing.es/sin-movimiento-no-siempre-prefiere-el-movimiento-reducido-trucos-css/>
- Argyle, A. (2020). prefers-reduced-motion: A veces, menos movimiento es más. web.dev. <https://web.dev/articles/prefers-reduced-motion?hl=es>